HRCIM-NTT: An Efficient Compute-in-Memory NTT Accelerator With Hybrid-Redundant Numbers

Xu Zhang, Yaodong Wei, Minghao Li, Jing Tian, *Member, IEEE*, and Zhongfeng Wang, *Fellow, IEEE*

***Abstract*— Recently, four NIST-approved Post-Quantum Cryptography (PQC) algorithms are selected to be standardized. Three of them are lattice-based cryptographic schemes and feature the number-theoretic transform (NTT) as the computing bottleneck compelling fast and low-power hardware implementations. In this work, a high-speed and power-efficient NTT accelerator is presented leveraging the compute-in-memory (CIM) technique with bottom-up optimizations. Firstly, a carry-free modular multiplication (CFMM) algorithm is proposed, which utilizes on-the-fly reduction and hybrid-redundant representation to optimize the butterfly unit operation, the cornerstone of NTT. Based on the optimized algorithm, an efficient butterfly unit in memory (BUIM) is developed by co-designing with SRAM circuit, which saves the memory access energy, decreases operation cycles, and obtains ultra-short critical path. Additionally, the data pattern of CIM array is also improved to avoid redundant memory read/write operations, which further reduces memory access overhead. Finally, a combination of pipelined operation flow and constant interstage data mapping strategy is employed to bestow the proposed hybrid-redundant CIM NTT (HRCIM-NTT) architecture with minimized computing cycles and reduced routing overhead. The implementation under 45nm CMOS technology demonstrates that HRCIM-NTT achieves the highest throughput and lowest latency among the existing CIM-based NTT accelerators.**

***Index Terms*— Post-quantum cryptography, number theoretic transform, polynomial multiplication, compute-in-memory, radix-4 booth multiplication, minimally redundant radix-4 addition.**

I. Introduction

QUANTUM computers have the potential to break the widely used public key encryption algorithms, such as Rivest Shamir Adlema (RSA) [1] and elliptic curve cryptography (ECC) [2], which rely on the difficulty of integer factoring and discrete logarithm problems, respectively. New cryptosystems that resist quantum attacks, also known as Post-Quantum Cryptography (PQC), are urgently needed. To this end, the National Institute of Standards and Technology (NIST) has initiated the process to standardize the PQC algorithms, including the Key Encapsulation Mechanisms (KEM) and the digital signature schemes [3]. The first group of PQC algorithms to be standardized includes CRYSTALS-Kyber (lattice-based KEM), CRYSTALS-DILITHIUM and Falcon (lattice-based digital signatures), and SPHINCS+ (symmetric-based digital signature). Most of them are lattice-based schemes, that provide both solid security and hardware-friendly way to build post-quantum cryptosystems [3], [4].

For lattice-based PQC algorithms, the polynomial multiplication is the key and time-consuming operation [5], [6]. Although NTT has reduced the time complexity of polynomial multiplication from $O(N^2)$ to $O(N \log N)$, it still remains a big computational bottleneck of the entire algorithm, due to the costly modular operations and the complex intermediate data transfer. Large amounts of solutions based on different platforms have been proposed to promote the performance of NTT. Graphics processing unit (GPU) possesses extraordinary computing power in terms of the matrix-multiply-and-add operations, but the control granularity of the single element level in NTT conflicts with the matrix mode of GPU. Moreover, interrupting computation frequently to access memory can severely damage the performance of GPU [7]. Besides, many solutions are based on Application-Specific Integrated Circuit (ASIC) and Field-Programmable Gate Array (FPGA), which make great progress in simplifying the modular operations and optimizing the dataflow of NTT but struggle to break the memory access bottleneck [8], [9], [10]. The read-write conflicts in the in-place NTT implementation on ASIC and FPGA result in the extra latency of memory access and make dataflow get bubbles [10]. The ping-pong memory access scheme has been proposed to enable reading and writing the intermediate coefficients in 1 cycle and reduce the latency of memory access [9], [11], but it requires double the memory capacity and leads to the higher power consumption. The layer merging technique is also introduced to reduce the loading and storage of intermediate data [12]. Overall, the above technologies take limited effects in eliminating the energy dissipation and latency overhead caused by the frequent intermediate data transfer, because their processing units and memory units are always separated.

As a new computation paradigm, compute-in-memory (CIM) [13], [14], [15], [16] performs the in-situ logic and arithmetical operations parallelly in the memory array where

Received 11 March 2024; revised 30 July 2024; accepted 12 September 2024. Date of publication 30 September 2024; date of current version 9 January 2025. This work was supported by the Ant Group under Grant CCF-AFSG RF20230201. This article was recommended by Associate Editor Z. Shi. *(Corresponding authors: Jing Tian; Zhongfeng Wang.)*

Xu Zhang and Minghao Li are with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China (e-mail: zhangxu96@smail.nju.edu.cn; minghaoli@smail.nju.edu.cn).

Yaodong Wei and Jing Tian are with the School of Integrated Circuits, Nanjing University, Suzhou 215163, China (e-mail: yaodongwei@smail.nju.edu.cn; tianjing@nju.edu.cn).

Zhongfeng Wang is with the School of Electronic Science and Engineering, Nanjing University, Nanjing 210023, China, and also with the School of Integrated Circuits, Sun Yat-sen University, Shenzhen 518107, China (e-mail: zfwang@nju.edu.cn).

Digital Object Identifier 10.1109/TCSI.2024.3463184

1549-8328 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

the data are stored. Therefore, CIM has the potential to eliminate the frequent intermediate data transfer between memory and processing units, achieving significant improvements in throughput and energy efficiency [17].

Various kinds of memory circuits have been used to implement the CIM architecture, such as static random access memory (SRAM), resistive random access memory (RRAM), dynamic random access memory (DRAM), phase change memory (PCM), magnetic random access memory (MRAM), and so on [18], [19]. Among them, SRAM-based CIM has been extensively explored in both academia and industry due to its high speed, excellent reliability, and compatibility with the advanced technologies [20]. For example, SRAM has been used to design compact and parallel in-memory NTT accelerators [21], [22], and almost all of these SRAM-based NTT accelerators utilize the bit-line computation to support bit-wise logical and arithmetical operations, by activating two wordlines simultaneously. Besides, they adopt either the bit-parallel [22], [23] or the bit-serial architecture [21], [24], [25], [26]. The bit-serial architecture provides massive bit-level parallelism and throughput at low area cost, but also leads to high latency. In addition, the bit-serial architecture requires more software or hardware design efforts to support the transposable data pattern. The bit-parallel architecture has the potential to achieve low latency and simple data pattern, but it suffers from the carry propagation problem [22], [24]. Both the bit-serial and the bit-parallel architectures have some common issues, such as the suboptimal partial product generation during multiplication, and redundant memory read/write operations, which will be detailed in Section II.

In this work, we mainly consider the two factors:

- The bit-parallel architecture requires less design efforts of SRAM circuit and software.
- The carry-free feature of hybrid-redundant addition [27] can effectively help to overcome the carry propagation problem of the bit-parallel architecture.

Thus, we design a bit-parallel compute-in-memory NTT accelerator with hybrid-redundant number representation, called HRCIM-NTT. In particular, the contributions of this paper are summarized as the following:

1) We propose a carry-free modular multiplication (CFMM) algorithm by developing a novel on-the-fly modular reduction method and incorporating the hybrid-redundant addition into Booth multiplication. Based on the optimized algorithm, we develop a novel in-memory butterfly unit, in which radix-4 Booth encoding is performed with low overhead to reduce operations by at least half compared to the previous CIM works [24], [28], [29]. Besides, the minimal redundant hybrid radix-4 addition (mrHY4A) is incorporated to complete the accumulation of partial products in a bit-parallel way, which fully breaks the carry propagation bottleneck and drastically helps to achieve low latency and high throughput.
2) We optimize the data pattern of CIM arrays. We choose only the constant twiddle factors to be stored in CIM arrays, which avoids frequently refreshing CIM arrays and further saves memory access overhead.
3) We introduce a compact constant interstage data mapping strategy to minimize the data routing overhead, and propose a lightweight pipeline with high hardware utilization to reduce latency and enhance throughput further.
4) We rigorously evaluate HRCIM-NTT under the 45nm CMOS process. The post-layout simulation shows that HRCIM-NTT achieves the latency of 0.18us and the throughput of 5.58KNTT/s, at the energy cost of only 16.73nJ per NTT.

The rest of this article is organized as follows. Section II provides the background and the motivation. Section III introduces the overall architecture of HRCIM-NTT. Section IV describes the hardware implementation of the proposed butterfly unit in memory (BUIM), which is the core component of HRCIM-NTT. Section V presents the proposed dataflow optimizations. Section VI shows the evaluation results and comparison with prior arts. Section VII discusses the potential limitations and future work. Section VIII concludes this article.

II. Background

A. Notation

In this paper, $R_q = Z_q[x]/(x^n + 1)$ represents a ring of polynomials, where n is a power of 2, and q is a prime number. In N-point and L-bit NTT, the expressions $Y + X \cdot W \bmod q$ and $Y - X \cdot W \bmod q$ correspond to one butterfly unit operation, where X and Y are two polynomial coefficients and W is a twiddle factor. Here, X, Y, and W are all L-bit unsigned integers and they can also be regarded as $(L+1)$-bit signed integers by zero extension in our design. X' and Y' are the minimally redundant radix-4 representations of X and Y, respectively. X' and Y' both have M digits, where $M = L/2 + 2$. W' is the pre-computed version of W, $W' = W \cdot 4^{M-1} \bmod q$. In our design, when referring to the twiddle factor, it means W' rather than W.

B. Number Theoretic Transform

Many lattice-based PQC algorithms are built on the hardness of solving the learning-with-errors (LWE) problems [30], where the challenge is to find a secret vector $\boldsymbol{s}$ given a pair of vectors $\boldsymbol{a}$ and $\boldsymbol{b}$ related by the equation.

$$\boldsymbol{b} = \boldsymbol{a} \cdot \boldsymbol{s} + \boldsymbol{e} \tag{1}$$

Here, $\boldsymbol{a} \in R_q^k$ is a random vector, $\boldsymbol{e} \in R_q^k$ is a noisy vector sampled from a Gaussian distribution. The operation $\boldsymbol{a} \cdot \boldsymbol{s}$ means a polynomial modular multiplication, which is over the polynomial ring $Z_q[X]/X^n + 1$ and executed in the NTT domain. NTT can reduce the computational complexity from $O(N^2)$ to roughly $O(N \log N)$. The Inverse Number Theoretical Transform (INTT) can be achieved by substituting the twiddle factor W in the NTT with its inverse element W^{-1}, and then multiplying the result by the scale factor N^{-1}. The concrete formula is described as below:

$$\begin{aligned} A_k = NTT(a_k) &= \sum_{n=0}^{N-1} a_n \omega_N^{nk} \bmod q \\ k &= 0, 1, \ldots, N-1 \\ a_n = INTT(A_n) &= \frac{1}{N} \sum_{k=0}^{N-1} A_k \omega_N^{-nk} \bmod q \\ n &= 0, 1, \ldots, N-1, \end{aligned} \tag{2}$$

With the NTT-domain representation, equation (1) can be represented as

$$\boldsymbol{b} = \mathrm{INTT}(\mathrm{NTT}(\boldsymbol{a}) \cdot \mathrm{NTT}(\boldsymbol{s})) + \boldsymbol{e}. \tag{3}$$

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

C. Redundant Numbers

In a conventional nonredundant radix-r system, a digit can range from 0 to $r-1$, and all numbers can be uniquely represented. Unlike that, a radix-r redundant signed-digit number system [27] employs a digit set $\{\overline{\beta}, \overline{\beta-1}, \cdots, \overline{1}, 0, 1, \cdots, \alpha\}$, where $\overline{\beta}$ denotes $-\beta, \beta \geq 1, \alpha \leq r-1$. In this number system, every number can take multiple representations, this redundancy enables the "carry-free" addition/subtraction which contributes to low latency or high speed. For example, the radix-2 redundant binary representation allows the digit set $\{-1, 0, +1\}$. Each digit is encoded by two unsigned binary bits, *i.e.,* $x_i = x_i^+ - x_i^-$, where $x_i^+, x_i^- \in \{0, 1\}$ and $x_i \in \{-1, 0, 1\}$. Thus one radix-2 redundant number can be encoded using two unsigned binary numbers, $X = X^+ - X^-$. Radix-2 hybrid-redundant numbers [27] are utilized to perform the carry-free addition and subtraction operations. Considering the addition of $S = X + Y$, where X and S are represented with radix-2 redundant numbers, and Y is represented with 2's complement. The addition is carried out in parallel for all bit positions, leading to an intermediate sum:

$$p_i = x_i + y_i = x_i^+ - x_i^- + y_i \tag{4}$$

The range of p_i lies in $\{-1, 0, 1, 2\}$. Obviously, at least two unsigned bits are required to represent p_i, and one of the two bits owns a negative weight. Thus we rewrite p_i as

$$p_i = 2t_i^+ - u_i^- \tag{5}$$

where t_i^+ and u_i^- represent the so-called transfer digit and interim sum, respectively. From another perspective, $2t_i^+$ is equivalent to s_{i+1}^+, the bit with positive weight in the next digit of the final sum S; while u_i^- is equivalent to s_i^-, the bit with negative weight in the current digit of S. s_{i+1}^+ and s_i^- can be computed with one plus-plus-minus adder (PPM) [31], which is shown in in Fig. 1.

$$s_{i+1}^+ = x_i^+ y_i + \overline{x_i^-}\left(x_i^+ + y_i\right) \tag{6}$$

$$s_i^- = x_i^+ \oplus y^i \oplus x_i^- \tag{7}$$

The overhead of one PPM is comparable with one full adder (FA), but PPM does not produce carry chain, so it can support parallel addition in a constant time independent of the word length of operands. PPM is the basic building block in redundant arithmetic. One minus-minus-plus adder (MMP) can be constructed with one PPM, just changing some wire connections as displayed in Fig. 1 (c). With FA, PPM, and MMP, various carry-free operations can be performed, such as redundant binary digit addition/subtraction, the maximally redundant hybrid radix-4 addition/subtraction, the minimally redundant hybrid radix-4 addition/subtraction, etc [27].

D. Compute-in-Memory Architecture

General processors and accelerators, including graphics processing units (GPUs), typically adhere to the von Neumann architecture shown in Fig. 2 (a), where the processing elements (PEs) and memory are independent entities, transferring data via the memory bus. The von Neumann architecture has provided a reliable and flexible foundation for continuous improvements in computational power over the past few decades [33]. However, the rise of data-intensive applications (*e.g.*, artificial intelligence (AI), cryptography, index search, and data analytics) has seen bottlenecks in this classical

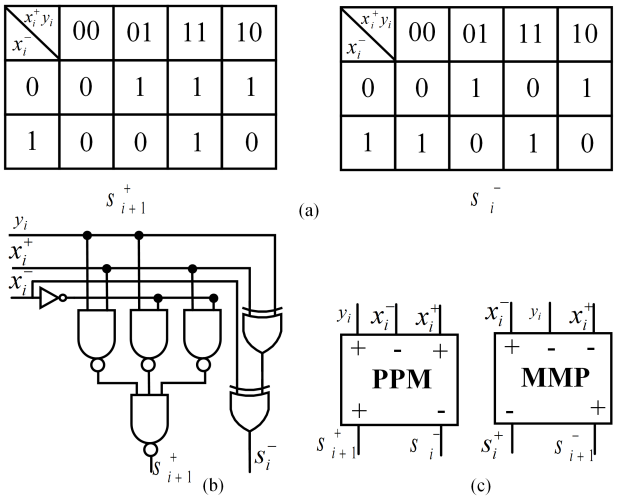


Fig. 1. PPM is the basic component supporting redundant arithmetic [31], [32]. (a) Karnaugh map. (b) gate-level netlist. (c) symbols.

computing architecture [23]. Studies show the data movement between memory and PEs takes 1000 times more time and 40 times more power compared to the arithmetic operations in PEs, which is the so-called memory wall [34].

In recent years, compute-in-memory (CIM) has been an attractive architecture that has the potential to break the memory wall. CIM eliminates the boundary between memory and PEs by integrating computing logic into memory, achieving significant bandwidth and energy efficiency improvements [16], [35], [36]. In the CIM architecture, computation and data access occur within the same clock cycle. This is different from the von Neumann architecture, where computation is performed in a cycle after data access [20]. Observing the differences in the circuit structures, Chang et al. classified CIM MACROs into near-memory computing (NMC) MACROs and in-memory computing (IMC) MACROs [20], [37].

As shown in Fig. 2 (b), the memory cell array in an NMC MACRO functions as a data storage circuit in the same manner as a typical memory. The digital or analog computing circuit is placed adjacent to the memory cell array, and the connection between them can be realized by customizing a multi-bit readout circuit [24] or directly reusing bit lines [38]. The external digital input and data accessed from the memory cell array are applied to the computing circuit, performing multi-bit logic operations, addition, multiplication, etc.

The memory cells in an IMC MACRO not only serve as storage units but also perform computation in analog or digital domain [20]. IMC usually involves redesigning the memory cells to support computation in situ. Special peripheral circuits are also required to activate multiple rows of SRAM cells and read out computational results simultaneously. Thus, IMC can offer maximum bandwidth and energy efficiency. On the other hand, IMC also introduces heavy redesigning effort and potential memory density issue [34].

The CIM architecture can also be classified from other perspectives. According to the applied signal types, CIM can be divided into analog compute-in-memory (ACIM) [39] and digital compute-in-memory (DCIM) [23], [24], [26], [40], [41]. ACIM offers higher energy efficiency but is susceptible to PVT variations, limiting its application requiring >8b precision [28]. In contrast, DCIM ensures full accuracy but

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

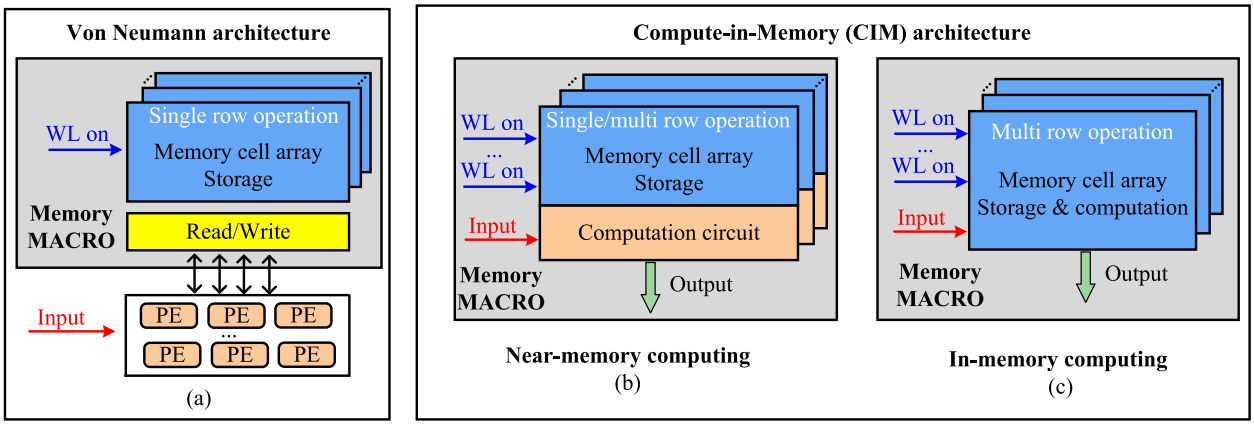


Fig. 2. Conceptual illustrations of the traditional von Neumann architecture and the CIM architecture [20], [37].

provides moderate energy efficiency [16]. Therefore, ACIM is promising in artificial intelligence of things (AIoT) [28] and DCIM is more suitable for high-precision applications such as cloud-edge AI inference, encryption, genome sequencing, etc [23], [28]. Additionally, based on the memory types, CIM can be categorized as volatile CIM [22] and nonvolatile CIM [39]. Nonvolatile CIM achieves high density and does not require reloading data when the system is turned on, but it suffers from low endurance and high write energy. Volatile CIM exhibits high speed and durability but has limited area density.

E. Related Work

Many compute-in-memory solutions for encryption applications have been proposed, including in-memory vector logic acceleration [25], in-memory matrix multiplication acceleration [42], in-memory polynomial multiplication acceleration [21], [22], [39], [43], etc. NTT is an efficient polynomial multiplication algorithm that plays an important role in post-quantum cryptography and homomorphic encryption. It has attracted wide attention from researchers in the field of compute-in-memory.

Different types of memory circuits are utilized to design compact and parallel in-memory NTT accelerators, especially the solutions based on RRAM and SRAM have continued to attract interest from academics. The advantage of RRAM-based CIM [39] lies in its high density and nonvolatility, making it suitable for very low-power applications. But the manufacturing process of RRAM is relatively immature, resulting in issues like reduced stability and limited endurance. By contrast, SRAM holds a long lifetime and high reliability, and it can be produced with the advanced logic process. These features make SRAM-based CIM more likely to achieve mass production in present ease [21].

Fig. 3 (a) shows how to enable the bitwise logic operations in SRAM. The bitwise AND logic is achieved by pre-charging the bit line (BL) to high and then enabling two words' wordlines simultaneously. If any bit in the two accessed SRAM cells is "0", BL will be discharged, which can present the result of the bitwise AND logic after being read out by a sense amplifier (SA).

Conversely, the bitwise NOR operation can be mirrored on the complementary bit line (BLB). Low-cost XOR logic in SRAM can be achieved by incorporating one NOR gate from the standard cell library, *i.e.,* XOR(A,B) = NOR(AND(A,B), NOR(A,B)). Based on the above property, both the bit-serial and the bit-parallel compute-in-memory architectures can be developed.

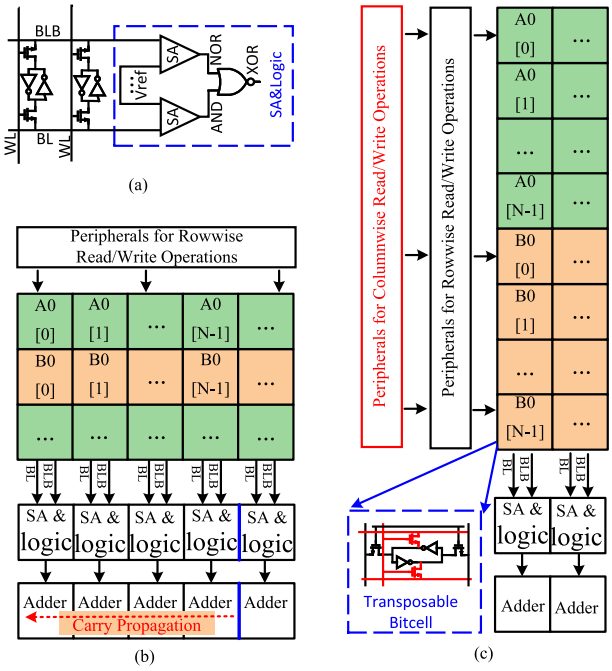


Fig. 3. Prior in-SRAM computing solutions for cryptographic applications. (a) In-SRAM bitline logic operation unit [24]. (b) Bit-parallel architecture suffering from the carry propagation problem [22], [24]. (c) Bit-serial architecture requiring either complex peripherals or additional programmer support [21], [24].

[25] constructs the in-SRAM logic units with the bit-parallel architecture, in which operands are stored with the traditional rowwise fashion and processed element-by-element, as described in Fig. 3 (b). Thus the logic operations up to 512 bits even can be completed within a single clock cycle. In [25], the researchers also implemented other near-memory cryptographic primitives to support various cryptography primitives including advanced encryption standard (AES), ECC, and Secure Hash Algorithm-3 (SHA-3). Performing the bit-parallel compute-in-memory architecture generally does not require modifying the foundry SRAM array [24], which reduces design costs, because its rowwise data mapping fashion for computing is fully compatible with the commercial

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

on-chip cache and SRAM chips. However, the bit-parallel logic units, if directly extended to the bit-parallel arithmetic units, will encounter the carry propagation problem as presented in Fig. 3 (b). This is especially serious for encryption applications, which often need to handle long bit-width operands [44], [45]. [22] performs the carry-save [46] addition to break the carry propagation bottleneck. Furthermore, the authors designed a hardware-aware modular reduction algorithm to reduce the complexity of NTT. However, it serially executes butterfly operations within each stage and lacks data flow optimization, leading to relatively low throughput.

To facilitate the implementation of bit-serial arithmetic operations in SRAM, [24] stores operands in a columnwise way (*i.e.,* transposed way) shown in Fig. 3 (c). This aligns all bits of a data element to the same bit line and allows each column to store at least one pair of operands (*e.g.*, addend and augend). Since the number of data elements in one SRAM array is typically much larger than the bit precision, activating two wordlines concurrently can enable many operations to be performed bit-by-bit instead of element-by-element, which brings enormous bit-level parallelism. However, it is at the cost of higher latency per operation. [21] extends the bit-serial compute-in-memory architecture to NTT domain, accelerating polynomial multiplication in a ring. They designed the near-memory modular reduction circuit co-optimized with the bit-serial arithmetic operations to reduce the complexity of modular arithmetic. The constant stage-to-stage routing is also utilized to simplify the dataflow. However, supporting the unconventional columnwise data mapping scheme needs to either customize programmer which increases software complexity, or design transposable memory cells and peripherals enabling data access in both horizontal and vertical directions.

The aforementioned approaches also encounter some common issues:

1) Modular multiplication is the most time-consuming part of NTT, but the prior CIM-based solutions mostly employ the simple shift-and-add method for multiplication, which results in relatively low computation efficiency. N-bit modular multiplication generates N partial products, and requires no less than N [22] and N^2 [21] clock cycles in the previous bit-parallel and bit-serial architectures, respectively. Both the quantity of partial products and the circuit delay are less optimal.
2) The polynomial coefficients are written into SRAM to perform in-SRAM computations by activating two rows simultaneously. However, in the execution process of lattice-based PQC algorithms such as Kyber, the polynomial coefficients are different for each round of NTT, leading to refreshing SRAM frequently. This dynamic storage fashion adds the burden of power consumption and bandwidth on the CIM arrays, sacrificing the advantage of reduced memory access brought by CIM [33], [47].

In summary, the previous bit-serial CIM architecture incurs long latency and complex hardware/software supports, and the previous bit-parallel CIM architecture has limited throughput and easily suffers from the carry propagation problem. They commonly have optimization space in terms of the core modular multiplication operator and the memory access pattern. Therefore, this work mainly focuses on implementing compact and fast modular multiplication in memory, reducing memory access overhead by optimizing the data storage pattern and data flow, and achieving efficient CIM-based NTT accelerator with high throughput and low latency.

III. Overall Architecture

HRCIM-NTT mainly consists of SRAM-based CIM arrays, which perform all modular arithmetic steps in and near memory to enable efficient NTT. The size of CIM arrays can be flexibly adjusted according to the number of points (N) and bit-width (L) of NTT. As displayed in Fig. 4, each CIM array includes multiple butterfly unit in memory (BUIM) modules, and BUIM is the core element of HRCIM-NTT. Each BUIM implements the basic butterfly operation, $Y + X \cdot W \bmod q$ and $Y - X \cdot W \bmod q$, optimized with radix-4 Booth algorithm and hybrid-redundant arithmetic. Every BUIM comprises L columns of SRAM cells, an attached partial product generator (PPG) as well as a hybrid-redundant arithmetic unit (HRAU). The modular multiplication is performed in a bit-parallel manner, whereas the addition and subtraction are digit-serial.

In the previous literature focusing on in-SRAM NTT accelerator, both the varying polynomial coefficients and the constant twiddle factors are stored in the CIM arrays. While NTT is performed, the polynomial coefficients are loaded from the input buffer and written into the CIM arrays, then the modular arithmetic can be completed, with many intermediate results written back to memory. This data storage pattern results in additional memory read-write overhead. Unlike the previous works, we only store the constant twiddle factors in the CIM arrays to avoid unnecessary read-write energy and latency. All the twiddle factors of a single NTT stage are stored in the same row of CIM arrays, thus activating one identical row of CIM arrays can enable all the butterfly operations in the same stage. For one N-point and L-bit NTT, there are totally $log_2 N$ stages, and each stage has $N/2$ twiddle factors. Therefore, it needs at least $log_2 N$ SRAM rows and each row has $L \cdot (N/2)$ bitcells to store the twiddle factors of a single NTT stage.

IV. Butterfly Unit in Memory

In this work, a compact and efficient butterfly unit in memory (BUIM) design is developed, based on algorithm-hardware co-optimization. A carry-free modular multiplication (CFMM) algorithm is proposed to complete $X \cdot W \bmod q$, by employing a novel compact modular reduction method and incorporating the minimally redundant hybrid radix-4 addition (mrHY4A) into the radix-4 Booth algorithm. Besides, the addition and subtraction in BUIM are carried out with digit-serial minimal redundant radix-4 addition/subtraction (mr4a/mr4s). Based on the Cooley–Tukey [48] butterfly operations performed in BUIM modules, the NTT procedure in Alg. 1 is completed in a pipelined manner under the scheduling of the top controller, rather than in a fully serial manner like the previous CIM-based accelerators [21], [22], [39], [43].

A. In-Memory Bit-Parallel Modular Multiplication

Our architecture makes use of hybrid-redundant number representation to enable the fast modular multiplication with reduced latency. Specifically, the twiddle factors are represented with 2's complement, and the polynomial coefficients X and Y are converted into the minimally redundant radix-4 representation, because it can be easily incorporated into the radix-4 Booth algorithm and reduce the iteration cycles of

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

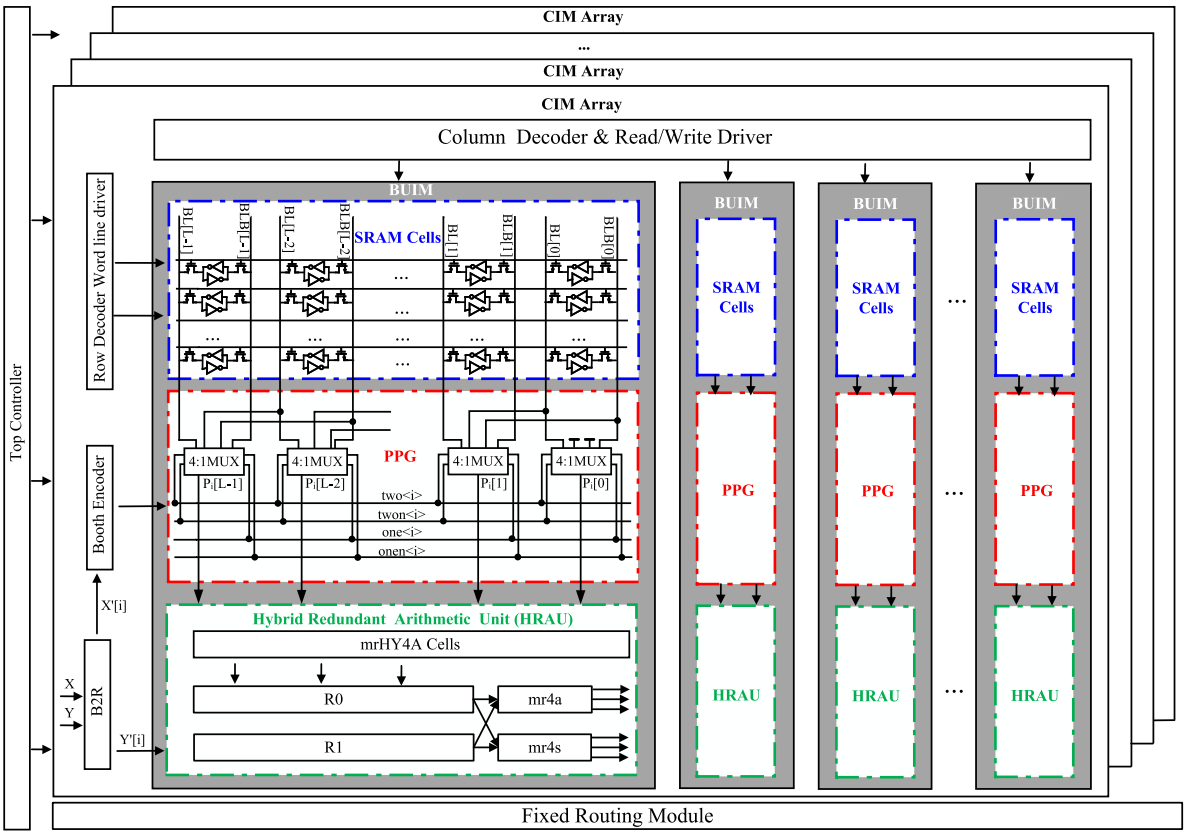


Fig. 4. Overall architecture of the proposed HRCIM-NTT accelerator. Abbreviations: partial product generator (PPG), butterfly unit in memory (BUIM), hybrid-redundant arithmetic unit (HRAU), binary-to-redundant conversion (B2R), and minimal redundant hybrid radix-4 adder (mrHY4A).

Algorithm 1 Iterative CT-NTT Based on The Proposed BUIM

Input: $a(x) \in \mathbb{Z}_q[x]/(x^n+1)$ in standard-order
Input: $\omega \in \mathbb{Z}_q[x]$ (precomputed table of 2n-th roots of unity, bit-reversed order)
Input: $n = 2^l, q$ (s.t. $q \equiv 1 \bmod n$)
Output: $A(x) \in \mathbb{Z}_q[x]/(x^n+1)$ in bit-reversed order

$t \leftarrow n/2$
for $(m = 1; m < n; m = 2m)$ **do**
  $k \leftarrow 0$
  for $(s = 0; s < m; s = s+1)$ **do**
    $W \leftarrow \omega[m+s]$
    for $(j = k; j < k+1; j = j+1)$ **do**
      $Y, X \leftarrow a[j], a[j+t]$
      $S \leftarrow$ **CFMM**(W, X)
      for $(i = 0; i < M; i = i+1)$ **do**
        $Z0[i] \leftarrow$ **mr4a**$(Y[i], S[i])$
        $Z1[i] \leftarrow$ **mr4s**$(Y[i], S[i])$
      end for
      $a[j], a[j+t] \leftarrow Z0, Z1$
    end for
    $k \leftarrow k+2t$
  end for
  $t \leftarrow t/2$
end for

serial addition by about half with regard to the redundant radix-2 representation. Input data X is recoded into X' with M digits, where $M = L/2+2$. Specifically,

$$X = \left(-2^L \cdot x[L] + \sum_{i=0}^{L-1} x[i] \cdot 2^i\right), \tag{8}$$

an equivalent minimally redundant radix-4 representation of X can be obtained by applying the method of 3-bit recoding:

$$X = \sum_{i=0}^{(L/2)+1} X'[i] \cdot 4^i, \tag{9}$$

where $X'[i]$ is the i-th digit of the redundant representation of X. $X'[i]$ is chosen from the set {-2, -1, 0, 1, 2} and encoded with three bits, d_i^{2-}, d_i^{+}, and d_i^{++}, which are respectively assigned weights of -2, 1, and 1, *i.e.,*

$$X'[i] = -2d_i^{2-} + d_i^{+} + d_i^{++}, \tag{10}$$

$$d_i^{2-} = x[2i+1], d_i^{+} = x[2i], d_i^{++} = x[2i-1]. \tag{11}$$

Here X should be extended by zero. Y is encoded into its redundant representation Y' in the same way.

Next, $W \cdot X \bmod q$ can be expressed as

$$W \cdot X \bmod q = \sum_{i=0}^{M-1} W \cdot X'[i] \cdot 4^i \bmod q. \tag{12}$$

This modular multiplication is generally treated in a most significant digit (MSD) first manner and it can be decomposed

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

into the following steps:

$$\begin{cases} step\ 1: P_i = W \cdot X'[M-1-i], \\ step\ 2: T = S_{i-1} \cdot 4 + P_i, \\ step\ 3: S_i = T \bmod q. \end{cases} \tag{13}$$

It can be completed by iterating Eq. (13) for M times with i ranging from 0 to $M-1$ and $S_{-1} = 0$. However, the left-shift ($S_{i-1} \cdot 4$) and the addition operations in step2 of Eq. (13) will both cause an increase in the width of intermediate sum (S_i), thereby making the reduction in step3 more complex. To optimize the hardware design, Eq. (12) is rewritten as:

$$\begin{aligned} W \cdot X \bmod q &= \sum_{i=0}^{M-1} (W \cdot 4^{M-1}) \cdot X'[i] \cdot 4^{i-(M-1)} \bmod q \\ &= ((\ldots((W' \cdot X'[0]) \cdot 4^{-1} + W' \cdot X'[1]) \cdot 4^{-1} + \\ &\ldots + W' \cdot X'[M-2]) \cdot 4^{-1} + W' \cdot X'[M-1]). \end{aligned} \tag{14}$$

Based on Eq. (14), the corresponding least significant digit (LSD) first-in modular multiplication steps can be given:

$$\begin{cases} step\ 1: P_i = W' \cdot X'[i], \\ step\ 2: T = S_{i-1} \cdot 4^{-1} + P_i, \\ step\ 3: S_i = T \bmod q, \end{cases} \tag{15}$$

where $W' = W \cdot 4^{M-1} \bmod q$, the index i ranges from 0 to $M-1$ and $S_{-1} = 0$. After performing the right-shift ($S_{i-1} \cdot 4^{-1}$) operation, $S_{i-1}[0]$ is abandoned and only the higher $(M-1)$ digits are remained, thus reducing bit width during accumulation. However, it may incur overflow/underflow issues, and extra additions are needed to compensate for the abandoned part. Based on the above consideration, we propose an on-the-fly modular reduction method:

$$\begin{cases} step\ 1: P_i = W' \cdot X'[i], \\ step\ 2: T = S_{i-1}[M-1:1] + P_i, \\ step\ 3: S_i = T + (S_{i-1}[0] \cdot 4^{-1} \bmod q). \end{cases} \tag{16}$$

The feasibility of the proposed reduction method lies in its ability to maintain the intermediate sum within the range of $(-4q, 4q)$. It can be proven by finding the upper and lower bounds of S_i.

We have known $X'[i], S_{i-1}[0] \in \{-2,-1,0,1,2\}$, $0 < W' \le q-1$. Note that the primes in NTT can always be written in the form of $q = \delta \cdot 2^e + 1$ [5], [49] and if $e \ge 2$, it is easy to derive that $4^{-1} \equiv -\delta \cdot 2^{e-2} = -\frac{q-1}{4} \bmod q$. Then we can get the range of terms in Eq. (16):

$$\begin{cases} S_{i-1}[M-1:1] = \frac{1}{4}(S_{i-1} - S_{i-1}[0]) \le \frac{1}{4}(S_{i-1}+2), \\ P_i = W' \cdot X'[i] \le 2(q-1), \\ S_{i-1}[0] \cdot 4^{-1} \bmod q \equiv S_{i-1}[0] \cdot (-\frac{q-1}{4}) \le \frac{q-1}{2}, \end{cases} \tag{17}$$

substituting Eq. (17) into Eq. (16), we observe that

$$S_i \le \frac{S_{i-1}}{4} + \frac{5}{2}(q-1) + \frac{1}{2}, \tag{18}$$

by iterating Eq. (18), we finally get

$$S_i \le \frac{10}{3}(q-1) + \frac{2}{3} + (\frac{1}{4})^i [S_0 - \frac{10}{3}(q-1) - \frac{2}{3}]. \tag{19}$$

Besides, $S_0 = P_0 \le 2(q-1)$, thus we can get the upper bound of S_i in Eq. (20), and its lower bound can be determined as $-4(q-1)$, through a similar derivation.

$$S_i \le \frac{10}{3}(q-1) + \frac{2}{3} - \frac{2}{3}(\frac{1}{4})^i [2(q-1)+1] < 4(q-1). \tag{20}$$

That means the reduced result is brought into the range $(-4q, 4q)$. Based on the compact reduction method, we can further develop a carry-free modular multiplication (CFMM) algorithm by utilizing radix-4 Booth encoding to generate the partial product (P_i) and incorporating minimally redundant hybrid radix-4 addition to complete the accumulation operations. The detailed procedure of the proposed CFMM is illustrated in Alg. 2.

Algorithm 2 Proposed Carry-Free Modular Multiplication (CFMM) Algorithm

Input: L-bit $q = \delta \cdot 2^e + 1$
Input: L-bit W' (pre-computed)
Input: M-digit X' (mr4)
Output: $S = W \cdot X' \bmod q$ (mr4)

```
S = 0                                          ▷ Initialize intermediate sum
l ← e/2 − 1
for (i = 0; i < M; i++) do
    two, twon, one, onen ← Booth_Encoder(X'[i])
    P_i ← PPG(two, twon, one, onen, W')
    for (j = 0; j < M − 1; j++) do
        {t_{j+1}^{++}, t_j^{2−}, t_j^{+}} ← mrHY4A(P_i[2j+1:2j], S[j+1])
    end for
    t_0^{++} ← P_i[L + 1]                      ▷ Plus sign bit
    for (k = 0; k < l; k++) do
        S[k] ← T[k]
    end for
    R ← S[0] · (−δ)
    for (k = l; k < M; k++) do
        p ← 2(k − l)
        {s_{k+1}^{++}, s_k^{2−}, s_k^{+}} ← mrHY4A(R[p+1:p], T[k])
    end for
    s_l^{++} ← R[msb]
end for
```

The in-memory implementation of CFMM is shown in Fig. 4 and Fig. 5. $X'[i]$ is fed into the Booth encoder placed near memory cells, in LSD first-in way, to obtain the coding signals $\{two[i], twon[i], one[i], onen[i], zero[i]\}$, according to the encoding rules in Table I. These coding signals control the partial product generator (PPG) to output one partial product per clock cycle, $P_i = W' \cdot X'[i]$. P_i is represented with $(L+2)$ bits using 2's complement, where the most significant bit is the sign bit. As shown in Table I, P_i behaves one of the five cases: W', $2W'$, their 1's complement $\overline{W'}$, $\overline{2W'}$ and 0. For the cases of $\overline{W'}$ and $\overline{2W'}$, we can easily get $-W'$ and $-2W'$ by adding the sign bit of P_i.

For the efficient in-memory implementation of CFMM, the compact PPG design is necessary to keep the regular layout and reasonable aspect ratio of the overall in-memory computing circuit. This goal is achieved by utilizing the cross-connected nodes of the SRAM cells to directly represent all the non-zero partial products. Specifically, each twiddle

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

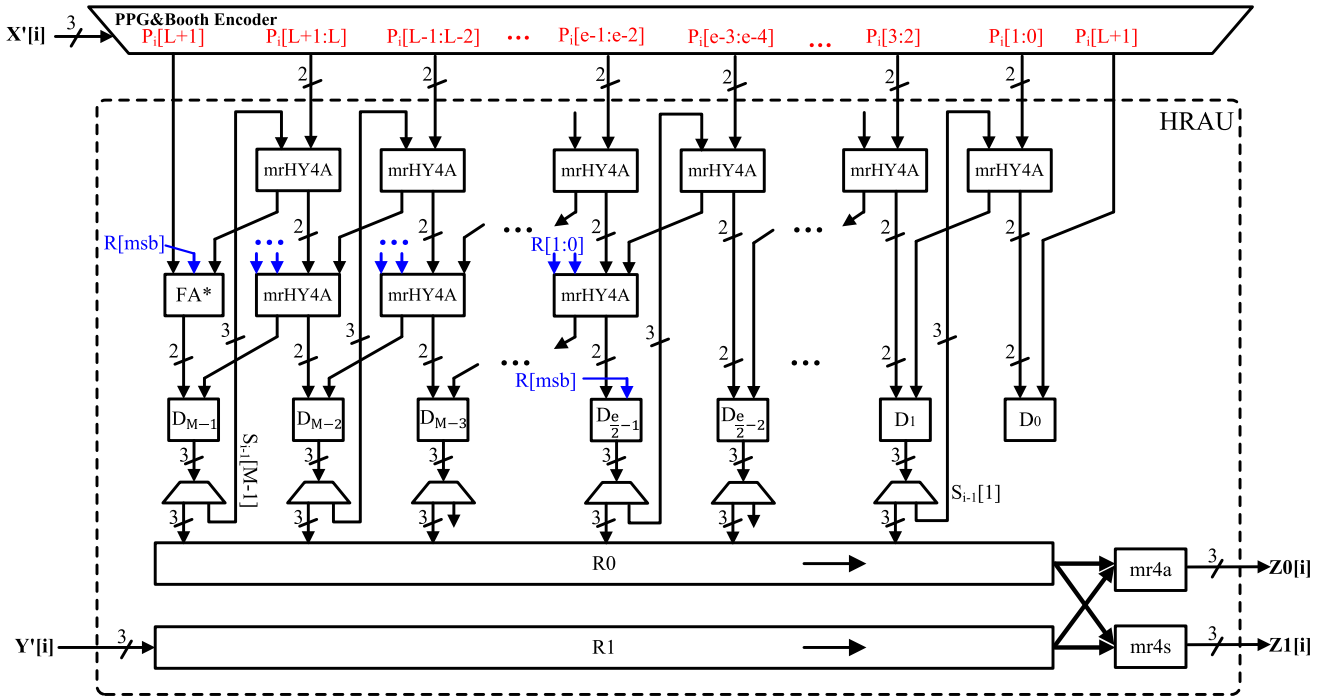


Fig. 5. Detailed diagram of the proposed hybrid-redundant arithmetic unit (HRAU), which overcomes the carry propagation issue in the bit-parallel architecture, enabling efficient modular multiplication with low latency.
*: The mrHY4A can be simplified to an FA.

TABLE I
TRUTH TABLE OF BOOTH ENCODER

x $[2i+1]$	x $[2i]$	x $[2i-1]$	X' $[i]$	P_i	two $[i]$	$twon$ $[i]$	one $[i]$	$onen$ $[i]$
0	0	0	0	0	0	0	0	0
0	0	1	+1	w'	0	0	1	0
0	1	0	+1	w'	0	0	1	0
0	1	1	+2	$2w'$	1	0	0	0
1	0	0	-2	$\overline{2w'}$	0	1	0	0
1	0	1	-1	$\overline{w'}$	0	0	0	1
1	1	0	-1	$\overline{w'}$	0	0	0	1
1	1	1	0	0	0	0	0	0

factor W', which serves as a multiplicand, is represented in 2's complement and stored in L consecutive SRAM cells in the same row, as shown in the dashed blue box of Fig. 4. Each SRAM row can store multiple twiddle factors. When a calculation cycle starts, the row address decoder activates the word line driver of the corresponding SRAM row to conduct the logic values stored in SRAM cells into the bit lines (BLs) and the bit-bar lines (BLBs). Thus the binary signals corresponding to W' will be reflected in the bit lines (BLs) while starting a calculation cycle. Due to the cross-coupled inverters in one SRAM cell, the binary signals of $\overline{W'}$ inherently exist in the bit-bar lines (BLBs). Similarly, $2W'$ and $\overline{2W'}$ can be reflected in the less significant BLs or BLBs, as shown in the dashed red box of Fig. 4. In this way, All the bit lines (BLs) and the bit-bar lines (BLBs) are utilized to achieve the left-shift and negation operations without any extra transistor overhead, leading to a very compact PPG circuit consisting of a series of 4-1 multiplexers. When the word line driver is activated, the Booth encoder receives a polynomial coefficient X and outputs the encoding signals simultaneously to control the PPG circuit to select the correct BLs or BLBs as the partial product signals. Thus, computation and data access can be performed within the same clock cycle.

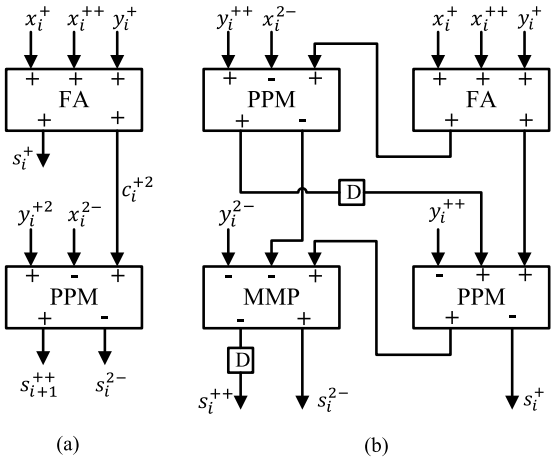


Fig. 6. Diagrams of (a) minimal redundant hybrid radix-4 adder cell (mrHY4A), and (b) minimal redundant radix-4 adder cell (mr4a).

The remaining operations of modular multiplication including the accumulation of partial products and modular reduction are completed by the hybrid-redundant arithmetic unit (HRAU) detailed in Fig. 5, The core element of HRAU is the minimal redundant hybrid radix-4 adder (mrHY4A) cell, which is composed of a PPM and a FA depicted in Fig. 6 (a).

HRAU employs mrHY4A cells to implement the accumulation of partial products in the bit-parallel way, of which the merit lies in the avoidance of carry chains, effectively reducing the critical path to one mrHY4A. The intermediate

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

sum is stored in M registers, $D_0 \sim D_{M-1}$ in Fig. 5. Note that there is no wire connection between the output of D_0 and any mrHY4A cell, i.e., the LSD is abandoned. It needs performing compensation according to the principle of our reduction method, and this is implemented by cascading the second-level mrHY4A cells in Fig. 5, to plus $S[0] \cdot (-\delta)$ (denoted as R in step 13 of Alg. 2). The compensation is only applied to the higher $M - e/2$ digits, and the lower $(e/2 - 1)$ digits are directly connected to the intermediate sum registers.

Thanks to the efficient in-memory implementation of the CFMM, HRCIM-NTT just needs $L/2 + 2$ clock cycles to perform modular multiplication with very short critical path. For comparison, modular multiplication occupies more than $(L + 1)^2$ clock cycles in the previous bit-serial in-memory implementations [21], [24], [50], and requires $62L - 8$ clock cycles in one prior bit-parallel in-memory implementation [22].

B. Near-Memory Digit-Serial Addition/Subtraction

In our design, the addition/subtraction in the current NTT stage and the multiplication in the next NTT stage run simultaneously, because we design a lightweight pipeline in Fig. 8 to optimize the data flow which will be elaborated in Section V. As described in Fig. 4 and Fig. 5, when a BUIM launches the butterfly operation, it loads coefficient Y' into right-shift registers $R1$ for addition/subtraction and feeds coefficient X' into the Booth encoder for multiplication, both in digit-serial. When the multiplication $W \cdot X' \bmod q$ is completed, the product S will be loaded into right-shift registers $R0$ in parallel. Next, the outputs of the two shift registers $R0$ and $R1$ are fed into the adder/subtracter in digit-serial to obtain the result of the corresponding butterfly unit. Different from mrHY4A used in multiplication, both inputs of the digit-serial adder/subtractor are represented by minimal redundant radix-4 numbers, which need minimal redundant radix-4 adder cell (mr4a) and minimal redundant radix-4 subtractor cell (mr4s) to perform the calculations. The schematic of mr4a is shown in Fig. 6 (b) and one mr4s can be formed by changing some wire connections of mr4a. The outputs of mr4a and mr4s are one digit of $Y + X \cdot W \bmod q$ and $Y - X \cdot W \bmod q$ respectively, which will be the new coefficients as the inputs of the next NTT stage via fixed routing (discussed in the next section). Due to applying the minimal redundant radix-4 addition, HRCIM-NTT requires $L/2 + 2$ clock cycles to perform addition/subtraction.

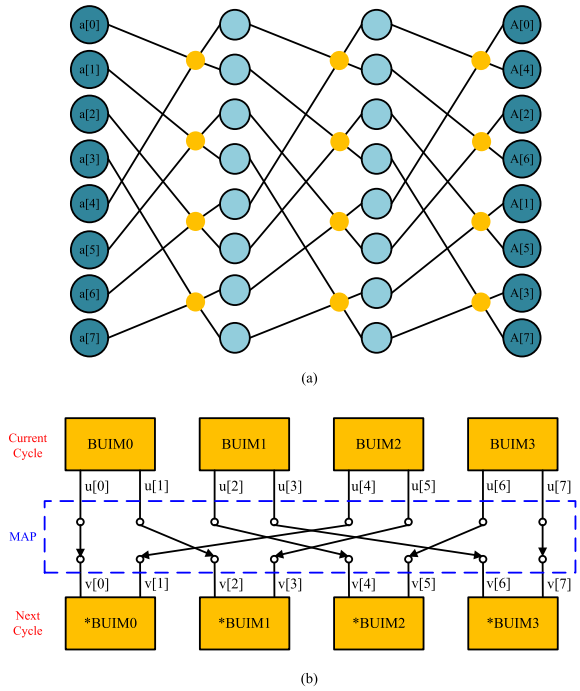


Fig. 7. HRCIM-NTT dataflow. (a) Data mapping strategy. (b) Hardware implementation with fixed routing.

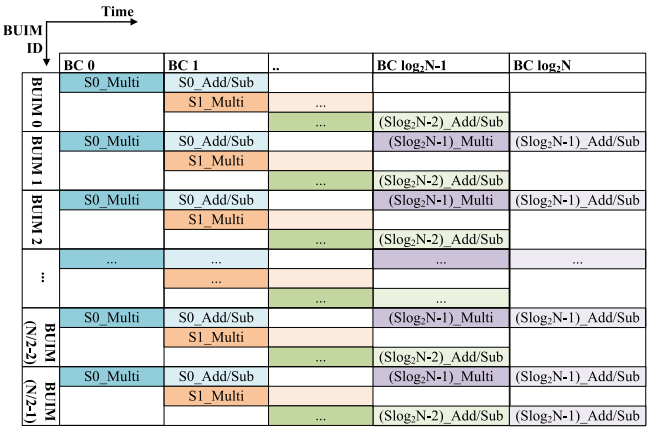


Fig. 8. HRCIM-NTT pipeline. BC 0 presents the first butterfly unit cycle which occupies $L/2+2$ clock cycles. S0_Multi and S0_Add/Sub present the multiplication and addition/subtraction in the first NTT stage, respectively.

V. Efficient NTT Dataflow

A. Compact Data Mapping Strategy

The traditional in-place NTT has complicated data routing that differs in each stage, making the mapping difficult with messy crossbars and extra control logic [51]. In HRCIM-NTT, we introduce a fixed routing strategy in Eq. (21) that ensures constant interstage data movement, enabling the compact NTT calculation process without complicated routing.

$$\begin{cases} v[2i] \leftarrow u[i], \\ v[2i+1] \leftarrow u[i + \dfrac{N}{2}]. \end{cases} \tag{21}$$

Here i represents the index of BUIM, from 0 to $(\frac{N}{2} - 1)$. Fig. 7 exemplifies an 8-point NTT. In Fig. 7 (a), each yellow cycle represents a Butterfly unit (BU), and data follows a predetermined path between the outputs and inputs of BUs. Fig. 7 (b) shows the corresponding hardware implementation of our scheme. For example, the output digits u[2] of BUIM1 and u[6] of BUIM3 are connected to v[4] and v[5] which will be fed into BUIM2 together next cycle.

MeNTT also utilizes one kind of constant interstage data mapping strategy, but it requires storing the outputs of the current NTT stage to memory first, and then takes $4 \cdot N$ clock cycles to map them into the memory address corresponding to the next NTT stage [21]. In our scheme, each HRAU outputs two digits of one butterfly unit operation per clock cycle. These two digits are mapped to the Booth encoders and HRAUs corresponding to the next NTT stage simultaneously, without occupying extra memory and clock cycles to store the intermediate data.

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

B. Lightweight Pipeline

Because HRCIM-NTT does not need extra memory access for reading and writing the intermediate data, it can be pipelined to allow simultaneous multiplication and addition/subtraction. Fig. 8 describes the operation flow of N-point and L-bit NTT. $L/2+2$ clock cycles constitute a butterfly unit cycle, noted as BC. During BC 0, The digits of X' are input to the BUIM one by one in $L/2+2$ clock cycles, completing the multiplication, $X' \cdot W \bmod q$. In the meantime, the digits of Y' are loaded to prepare for the addition/subtraction, while the mr4a and mr4s are temporarily in an idle state. During BC 1, the mr4a and mr4s start working to output two digits of $Y' \pm X' \cdot W \bmod q$ per clock cycle. The output digits are mapped to the next NTT stage to perform the multiplication, according to the mapping strategy in Fig. 7. In this way, the addition/subtraction of the current NTT stage and the multiplication of the next NTT stage are simultaneously carried out in the following BCs. During the last BC, only the mr4a and mr4s work to output the final NTT results.

Because the bit-parallel multiplication and the bit-serial addition/subtraction both require $L/2+2$ clock cycles, the pipeline has no idle state and the hardware utilization can reach 100% except for the first and last BCs. On average, the butterfly operations of a NTT stage need $L/2+2$ clock cycles. For N-point and L-bit NTT, the pipeline contains $(log_2 N+1)$ BCs, so the total number of clock cycles of one whole NTT is just $(log_2 N+1) \cdot (L/2+2)$.

VI. Evaluation

In this section, we analyze the performance indicators of the proposed HRCIM-NTT, such as throughput, latency, and energy. All the circuits are realized with 45-nm CMOS technology. We use Cadence tools to create PPG modules and an SRAM bank with four SRAM subarrays based on the SRAM circuit generated with an open-source memory compiler [52], which is widely used to evaluate the CIM architecture design [22], [42], [45], [53], and each subarray contains 16×192 cells. This configuration is fully compatible with CRYSTAL-Kyber [9], [10], [11]. In CRYSTAL-Kyber, each NTT stage needs 64 12-bit twiddle factors which can be distributed in the same row of the four subarrays. The remaining hardware modules are digital circuits coded with System Verilog and synthesized with Synopsys Design Compiler. The placement and route flow is done in Cadence Innovus. The Spice simulation is carried out to verify the correctness and estimate the power and timing performances of the overall system accurately. Table III shows a comparison between HRCIM-NTT and previous FPGA, ASIC, software, and CIM designs.

A. Cycle Cost, Latency, and Speed

As shown in Table II, the modular multiplication in HRCIM-NTT just needs $L/2+2$ clock cycles, which mainly benefits from the following optimizations: HRCIM-NTT applies radix-4 Booth algorithm and bit-parallel mrHY4A to reduce iteration cycles. Additionally, the proposed on-the-fly modular reduction does not require extra clock cycles. The multiplication cycle cost in HRCIM-NTT is only approximately 1% of BP-NTT [22]. That's also much lower than the multiplication cycles of the ones utilizing the bit-serial arithmetic such as MeNTT [21], Neural Cache [24], and CryptoPIM [50], where the time complexity of multiplication is $O(L^2)$. Besides, thanks to the carry-free property of mrHY4A, the critical path requires only 1.33ns for the implementation in 45nm, mainly contributed by the butterfly unit. HRCIM-NTT applies digit-serial mr4a and mr4s to perform addition and subtraction in $L/2+2$ clock cycles, which makes a trade-off between hardware overhead and latency.

TABLE II
Comparison on Cycles of Arithmetic Operations Performed by Different CIM-Based Solutions

	Multiplication	Addition	Subtraction
This Work (HRCIM-NTT)	$\boldsymbol{L/2+2}$	$\boldsymbol{L/2+2}$	$\boldsymbol{L/2+2}$
MeNTT [21]	$(L+1)^2$	$2(L+1)$	$3(L+1)$
BP-NTT [22]	$62L-8$	$8L-8$	$8L-5$
Neural Cache [24]	L^2+5L-2	$L+1$	/
CryptoPIM [50]	$6.5L^2-11.5L+3$	$6L+1$	$7L+2$

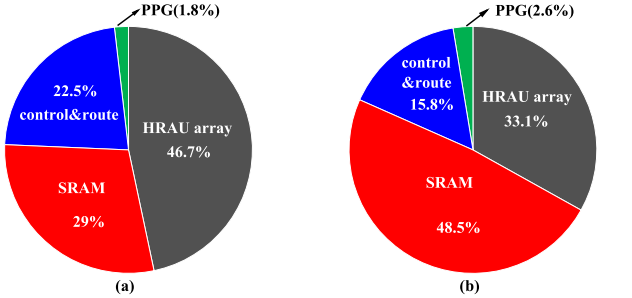


Fig. 9. Energy and area breakdown of HRCIM-NTT. (a) Energy. (b) Area.

The previous bit-serial CIM works usually convert subtraction into addition and require extra $N+1$ clock cycles for 2's complement calculation. However, HRCIM-NTT does not require dedicated processing for the sign, because a redundant number can be itself either positive or negative, this also helps to reduce latency.

Owing to the inserted one-stage pipeline, multiplication, addition, and subtraction run simultaneously in our design, which saves the entire NTT execution cycles by about 50%. In contrast, the previous CIM-based NTT accelerators do not optimize the dataflow so that they can only serially execute multiplication, addition, and subtraction in butterfly units, hindering the throughput of NTT.

Based on the above optimizations, HRCIM can run at 714MHz and achieve the latency of 0.18us and the throughput of 5.58KNTT/s, which is much better than many solutions based on CIM, ASIC, and FPGA shown in Table III.

B. Energy Consumption and Hardware Cost

Fig. 9 shows the energy and area breakdown of HRCIM-NTT and Fig. 10 details the layout micrograph of two adjacent BUIMs in HRCIM-NTT. HRCIM-NTT reduces hardware cost and power consumption in both computation and memory access aspects.

At first, radix-4 Booth multiplication is cooperated into SRAM-based CIM to reduce operations by approximately half, which helps to reduce much computation energy. During performing multiplication, HRCIM-NTT reuses the SRAM circuits to generate multiples of twiddle factors with costless shift and negation operations. This approach results in a compact in-memory PPG module with low power consumption, requiring approximately 25% transistors compared to the PPG

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON TABLE WITH STATE-OF-THE-ART NTT ACCELERATORS ON A 256-POINT POLYNOMIAL

	This work	MeNTT [21]	BP-NTT [22]	RM-NTT [39]	CryptoPIM [50]	LEIA [54]	VPQC [55]	SA_NTT [56]	KyberMat [57]	CPU [50]
Method	**In-SRAM**	In-SRAM	In-SRAM	In-RRAM	In-RRAM	ASIC	ASIC	FPGA	FPGA	software
Technology	**45nm**	65nm	45nm	28nm	45nm	40nm	28nm	-	-	-
Bitwidth	**12**	14	16	14	16	14	12	16	12	16
Frequency (MHz)	**714**	151	3.8K	400	909	300	300	184	222	2K
Latency (μs)	**0.18**	23	61.9	0.28	68.7	0.53	0.137	21.6	0.67	85
Throughput (KNTT/s)	**5.58K**	42	258.6	3.57K	553.3	1.88K	7.3K	46	4.26K	11.8
Energy (nJ/NTT)	**16.73**	144	34	145	2.6K	31	21.4	21.5k	-	570K
†Normalized Energy (nJ/NTT)	**1.4**	6.5	2.1	17.6	162.5	2.5	3	-	-	-
Area ($\mu m^2 \times 10^3$)	**123**	360	63	*101	*152	*914	-	-	-	-
†Normalized Area ($\mu m^2 \times 10^3$)	**10.3**	12.6	3.9	*20.6	*9.5	*87	-	-	-	-

† The Energy and Area are normalized to 1 bit in the 45nm process.
* These works do not provide area overhead explicitly. We estimate their area by referring to work [22].

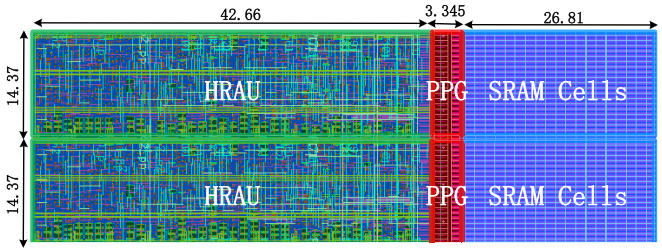


Fig. 10. Layout micrograph of two adjacent BUIMs.

used in conventional all-digital Booth multipliers [29], [58]. All PPG modules only account for 1.8% of the total energy. BP-NTT performs another kind of costless shift operation, but it achieves less energy saving, because BP-NTT utilizes the simple shift-and-add to accomplish multiplication so that it has to handle about twice the partial sums of our design. Except for the partial product generation performed in memory, the near-memory HRAU completes all the rest computations in a BUIM. HRAU applies the folded architecture to save hardware costs, making it easy to fit the pitch of memory as shown in Fig. 10 and minimizing the impact on storage density. Besides, we simplified the process of modular reduction by exploiting the form of the primes in NTT and the properties of radix-4 hybrid-redundant arithmetic. With the above optimizations, we make the near-memory HRAU a low-cost design. As the most computation-intensive parts, HRAUs occupy 33.1% of total area and consume 46.7% of total energy.

In terms of memory access, HRCIM-NTT reduces a lot of redundant memory read/write operations. HRCIM-NTT only stores the constant twiddle factors to avoid refreshing the memory frequently performed in most previous CIM designs. Besides, the interim data also does not need to be written back to memory. The above optimized data pattern decreases the memory access overhead significantly, so the SRAM circuits only require less than 30% of the overall energy, which is about half of MeNTT. RM-NTT [39] also only stores twiddle factors, but it uses an unfolded vector-matrix multiplication (VMM) approach to achieve maximal parallelism during NTT and INTT, so it needs $N^2 \cdot L$ bitcells to store the original twiddle factor matrix. In comparison, HRCIM-NTT only occupies the memory space of $(log_2 N \cdot N/2) \cdot L$ bitcells.

With these improvements, HRCIM-NTT just requires 16.73nJ per NTT, which is very low among the existing CIM-based NTT accelerators, as shown in Table III.

C. Extendibility Analysis

HRCIM-NTT is easily extended to support different NTT configurations of modulus, bit width, and point. To handle a higher bit width, it is necessary to increase the bit cells in each row of BUIMs and the adders in HRAUs. Profiting from the carry-free property of hybrid-redundant arithmetic, our design can maintain ultra short critical path across various bit widths, which also indicates its potential in encryption applications with ultra-high bit width, such as fully homomorphic encryption (FHE). As analysed in section V, the clock cycles occupied by each butterfly unit cycle and the adders in each HRAU both present a linear correlation with the bit width. Therefore, the latency and the energy exhibit approximately linear and quadratic growth with the extension of bit width in Fig. 11. An increase in points of NTT requires more stages and more twiddle factors in each stage, which enlarges the size of CIM array in both the horizontal and vertical directions, resulting in the nonlinear trends in Fig. 12. In this case, the parasitic capacitance of word lines and the routing length between stages can be reduced by folding a large array into several smaller subarrays. Adjusting the modulus not only affects the size of the SRAM array but also requires an adjustment in the number of adders in the second-level compensation circuit of the HRAU.

VII. DISCUSSION

A. Overheads Due to Hybrid-Redundant Representation

Though the hybrid-redundant representation greatly helps to achieve high speed, it is necessary to investigate the extra overhead compared to the conventional binary format. Regarding

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

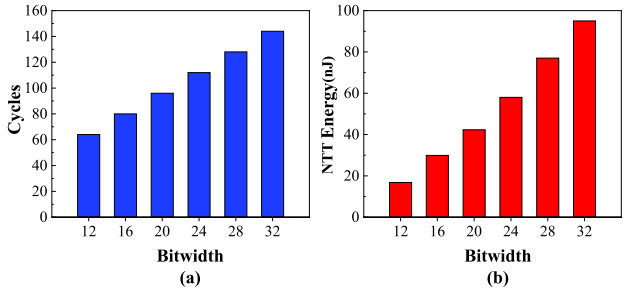


Fig. 11. HRCIM-NTT energy and clock cycles for different bitwidth at N = 256. (a) clock cycles. (b) energy.

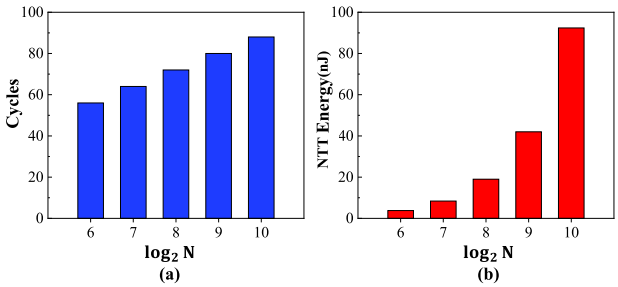


Fig. 12. HRCIM-NTT energy and clock cycles for different points at L = 12b. (a) clock cycles. (b) energy.

L-bit modular multiplication, $M-1(M=L/2+2)$ minimal redundant hybrid radix-4 adders (mrHY4As) are required to complete the accumulation of the partial products when using the hybrid-redundant radix-4 representation. The overhead of these mrHY4As is comparable to $L+2$ full adders. In contrast, at least L full adders are required to construct a ripple carry adder to complete the accumulation of the partial products when using the conventional format. That is to say, the hybrid-redundant radix-4 representation leads to the extra overhead of about two full adders compared to the conventional format. But it is fully worthwhile due to the following reasons: The critical path in $M-1(M=L/2+2)$ mrHY4As is only the delay of two full adders, which is independent of bit-width. But the critical path in a ripple carry adder is the delay of about L full adders, which increases linearly with bit-width. Thus, much more pipeline registers are needed to reduce the critical path when using the conventional format. In addition, the process of modular reduction is also simplified in our design taking advantage of the hybrid-redundant representation.

Another potential overhead comes from the necessary format conversion. In a PQC cryptosystem, input and output numbers typically use regular binary representation. Consequently, binary-to-redundant conversion (B2R) becomes necessary for input numbers, while redundant-to-binary conversion (R2B) is required for computation results. B2R mathematically involves simple assignments, as shown in Eq. 11. Thus, B2R is really easy to achieve and does not increase any circuit resources or latency. As for R2B, all the intermediate data can be maintained in redundant form. In the end, the redundant results can be recovered to regular binary format. Each R2B module receives a redundant digit $\{x_i^{2-}, x_i^{+}, x_i^{++}\}$ as input signal and outputs two binary bits $\{s_i^{2+},\ s_i^{+}\}$ and a propagated carry bit $\{c_{i+1}\}$. It can be implemented in the following way:

$$-2x_i^{2-}+x_i^{+}+x_i^{++}-c_i=-4c_{i+1}+2s_i^{2+}+s_i^{+} \tag{22}$$

$$\begin{cases}\{t_i^{2-}, s_i^{+}\}=\mathrm{MMP}(x_i^{++}, x_i^{+}, c_i)\\ \{c_{i+1}, s_i^{2+}\}=\mathrm{PPM}(x_i^{2-}, x_i^{+}, t_i^{2-})\end{cases} \tag{23}$$

Eq. 23 shows an R2B module can be constructed by cascading an MMP and a PPM, whose overhead is just comparable to two full adders. Considering the numerous intermediate calculations in a PQC cryptosystem, the extra latency cost and area overhead introduced by R2B can be negligible compared to the entire cipher operation [59].

B. Future Work

While our scalable CIM scheme is capable of supporting different NTT parameters, including prime width and polynomial length, the hardware implementation is specifically optimized for a single parameter set, such as that used in the Kyber algorithm. Therefore, our work lacks on-chip reconfigurability compared to the implementations like MeNTT and BP-NTT which adapt to various NTT parameters through on-chip programming or configuration. Nevertheless, this disadvantage is not a significant drawback for our intended use cases in respect that the edge-computing devices usually support only one or two cryptographic algorithms. Despite this, we recognize the value of on-chip reconfigurability and acknowledge it as an attractive area for future research.

Except for dominating time and energy in PQC algorithms, NTT is also the main performance bottleneck in FHE. NTT used in FHE involves large integers and high polynomial degrees. Although the applied hybrid-redundant arithmetic can help to maintain a low critical path in a large integer multiplier, extending the architecture of HRCIM into FHE may still face challenges in on-chip data storage and throughput. We will propose solutions to address these issues in our future work.

VIII. Conclusion

In this paper, we have proposed, developed, and validated HRCIM-NTT, one novel in-SRAM NTT accelerator with low latency, high throughput, and low power. The core component of HRCIM-NTT is the butterfly unit in memory (BUIM). In BUIM, a novel carry-free modular multiplication (CFMM) algorithm is implemented in SRAM using the hybrid-redundant arithmetic, enabling compact modular multiplication with high speed and low power. Additionally, The data storage pattern of CIM array is also optimized to eliminate redundant memory access. Finally, a constant inter-stage data transfer scheme and an efficient pipeline are performed in memory, reducing the data routing overhead and improving the throughput. The implementation under 45nm CMOS technology has verified that HRCIM-NTT achieves the highest throughput and lowest latency among the existing CIM-based NTT accelerators.

References

[1] D. Boneh et al., "Twenty years of attacks on the RSA cryptosystem," *Notices AMS*, vol. 46, no. 2, pp. 203–213, 1999.

[2] J. W. Bos, J. A. Halderman, N. Heninger, J. Moore, M. Naehrig, and E. Wustrow, "Elliptic curve cryptography in practice," in *Proc. Int. Conf. Financial Cryptogr. Data Secur. (FC)*. Christ Church, Barbados: Springer, Mar. 2014, pp. 157–175.

[3] L. Malina, P. Dobias, J. Hajny, and K.-K.-R. Choo, "On deploying quantum-resistant cybersecurity in intelligent infrastructures," in *Proc. 18th Int. Conf. Availability, Rel. Secur.*, Aug. 2023, pp. 1–10.

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

[4] X. Chen, B. Yang, S. Yin, S. Wei, and L. Liu, "CFNTT: Scalable radix-2/4 NTT multiplication architecture with an efficient conflict-free memory mapping scheme," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2022, no. 1, pp. 94–126, 2022.

[5] M. Li, J. Tian, X. Hu, and Z. Wang, "Reconfigurable and high-efficiency polynomial multiplication accelerator for CRYSTALS-kyber," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2540–2551, Aug. 2023.
Li, M., Tian, J., Hu, X. and Wang, Z.,Aug.2023. Reconfigurable and high-efficiency polynomial multiplication accelerator for crystals-kyber. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 42(8), pp.2540-2551.

[6] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-KYBER on FPGA," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2021, pp. 328–356, Feb. 2021.

[7] L. Wan et al., "A novel high-performance implementation of CRYSTALS-Kyber with AI accelerator," in *Proc. Eur. Symp. Res. Comput. Secur.* Cham, Switzerland: Springer, 2022, pp. 514–534.

[8] S. Roy, F. Vercauteren, N. Mentens, D. Chen, and I. Verbauwhede, "Compact ring-LWE cryptoprocessor," in *Proc. Int. Workshop Cryptograph. Hardw. Embedded Syst.*, 2014, pp. 371–391.

[9] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "Instruction-set accelerated implementation of CRYSTALS-Kyber," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 11, pp. 4648–4659, Nov. 2021.

[10] Z. Chen, Y. Ma, T. Chen, J. Lin, and J. Jing, "High-performance area-efficient polynomial ring processor for CRYSTALS-Kyber on FPGAs," *Integration*, vol. 78, pp. 25–35, May 2021.

[11] C. Zhang et al., "Towards efficient hardware implementation of NTT for kyber on FPGAs," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2021, pp. 1–5.

[12] L. Zhao, J. Zhang, J. Huang, Z. Liu, and G. Hancke, "Efficient implementation of Kyber on mobile devices," in *Proc. IEEE 27th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Dec. 2021, pp. 506–513.

[13] S. Yu, H. Jiang, S. Huang, X. Peng, and A. Lu, "Compute-in-memory chips for deep learning: Recent trends and prospects," *IEEE Circuits Syst. Mag.*, vol. 21, no. 3, pp. 31–56, 3rd Quart., Aug. 2021.

[14] L. Ding, S. Bian, and J. Zhang, "PIMA-LPN: Processing-in-memory acceleration for efficient LPN-based post-quantum cryptography," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.

[15] M. Lee et al., "FeFET-based low-power bitwise logic-in-memory with direct write-back and data-adaptive dynamic sensing interface," in *Proc. ACM/IEEE Int. Symp. Low Power Electron. Design*, Aug. 2020, pp. 127–132.

[16] Z. Zhang et al., "From macro to microarchitecture: Reviews and trends of SRAM-based compute-in-memory circuits," *Sci. China Inf. Sci.*, vol. 66, no. 10, Oct. 2023, Art. no. 200403.

[17] A. Jaiswal, A. Agrawal, M. F. Ali, S. Sharmin, and K. Roy, "I-SRAM: Interleaved wordlines for vector Boolean operations using SRAMs," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 12, pp. 4651–4659, Dec. 2020.

[18] M. Ali, S. Roy, U. Saxena, T. Sharma, A. Raghunathan, and K. Roy, "Compute-in-memory technologies and architectures for deep learning workloads," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 11, pp. 1615–1630, Nov. 2022.

[19] F. Liu et al., "Bit-transformer: Transforming bit-level sparsity into higher preformance in ReRAM-based accelerator," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Nov. 2021, pp. 1–9.

[20] C.-J. Jhang, C.-X. Xue, J.-M. Hung, F.-C. Chang, and M.-F. Chang, "Challenges and trends of SRAM-based computing-in-memory for AI edge devices," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 68, no. 5, pp. 1773–1786, May 2021.

[21] D. Li, A. Pakala, and K. Yang, "MeNTT: A compact and efficient processing-in-memory number theoretic transform (NTT) accelerator," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 5, pp. 579–588, May 2022.

[22] J. Zhang, M. Imani, and E. Sadredini, "BP-NTT: Fast and compact in-SRAM number theoretic transform with bit-parallel modular multiplication," 2023, *arXiv:2303.00173*.

[23] W. A. Simon, Y. M. Qureshi, M. Rios, A. Levisse, M. Zapater, and D. Atienza, "BLADE: An in-cache computing architecture for edge devices," *IEEE Trans. Comput.*, vol. 69, no. 9, pp. 1349–1363, Sep. 2020.

[24] C. Eckert et al., "Neural cache: Bit-serial in-cache acceleration of deep neural networks," in *Proc. ACM/IEEE 45th Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2018, pp. 383–396.

[25] Y. Zhang, L. Xu, Q. Dong, J. Wang, D. Blaauw, and D. Sylvester, "Recryptor: A reconfigurable cryptographic cortex-M0 processor with in-memory and near-memory computing for IoT security," *IEEE J. Solid-State Circuits*, vol. 53, no. 4, pp. 995–1005, Apr. 2018.

[26] J. Wang et al., "A 28-nm compute SRAM with bit-serial logic/arithmetic operations for programmable in-memory vector computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 76–86, Jan. 2020.

[27] K. K. Parhi, *VLSI Digital Signal Processing Systems: Design and Implementation*. Hoboken, NJ, USA: Wiley, 2007.

[28] F. Tu et al., "ReDCIM: Reconfigurable digital computing-in-memory processor with unified FP/INT pipeline for cloud AI acceleration," *IEEE J. Solid-State Circuits*, vol. 58, no. 1, pp. 243–255, Jan. 2023.

[29] W.-C. Yeh and C.-W. Jen, "High-speed booth encoded parallel multiplier design," *IEEE Trans. Comput.*, vol. 49, no. 7, pp. 692–701, Jul. 2000.

[30] M. ElGhamrawy et al., "From MLWE to RLWE: A differential fault attack on randomized & deterministic dilithium," *IACR Trans. Cryptograph. Hardw. Embedded Syst.*, vol. 2023, pp. 262–286, Aug. 2023.

[31] A. Guyot, Y. Herreros, and J.-M. Müller, "JANUS, an on-line multiplier/divider for manipulating large numbers," in *Proc. 9th Symp. Comput. Arithmetic*, vol. 495, Sep. 1989, pp. 106–111.

[32] H. R. Srinivas and K. K. Parhi, "High-speed VLSI arithmetic processor architectures using hybrid number representation," *J. VLSI Signal Process. Syst. Signal, Image Video Technol.*, vol. 4, nos. 2–3, pp. 177–198, May 1992.

[33] A. Agrawal, A. Jaiswal, C. Lee, and K. Roy, "X-SRAM: Enabling in-memory Boolean computations in CMOS static random access memories," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 65, no. 12, pp. 4219–4232, Dec. 2018.

[34] D. Fujiki, X. Wang, A. Subramaniyan, and R. Das, *In-/Near-Memory Computing*. Cham, Switzerland: Springer, 2021.

[35] A. Guo et al., "34.3 a 22nm 64kb lightning-like hybrid computing-in-memory macro with a compressed adder tree and analog-storage quantizers for transformer and CNNs," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 570–572.

[36] X. Si et al., "A 28 nm 16-kb sign-extension-less digital-compute-in-memory macro with extension-friendly compute units and accuracy-adjustable adder-tree," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, pp. 1–5, 2024.

[37] J.-M. Hung, C.-J. Jhang, P.-C. Wu, Y.-C. Chiu, and M.-F. Chang, "Challenges and trends of nonvolatile in-memory-computation circuits for AI edge devices," *IEEE Open J. Solid-State Circuits Soc.*, vol. 1, pp. 171–183, 2021.

[38] B. Yan et al., "A 1.041-Mb/MM 2 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 188–190.

[39] Y. Park, Z. Wang, S. Yoo, and W. D. Lu, "RM-NTT: An RRAM-based compute-in-memory number theoretic transform accelerator," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 8, pp. 93–101, 2022.

[40] S. Jeloka, N. B. Akesh, D. Sylvester, and D. Blaauw, "A 28 nm configurable memory (TCAM/BCAM/SRAM) using push-rule 6T bit cell enabling logic-in-memory," *IEEE J. Solid-State Circuits*, vol. 51, no. 4, pp. 1009–1021, Apr. 2016.

[41] W. A. Simon, "In-memory hardware and architectural extensions for workloads acceleration," Doctoral dissertation, EPFL, Lausanne, p. 136, Sep. 2022, doi: 10.5075/epfl-thesis-9261. [Online]. Available: https://infoscience.epfl.ch/handle/20.500.14299/190236

[42] D. Reis, H. Geng, M. Niemier, and X. S. Hu, "IMCRYPTO: An in-memory computing fabric for AES encryption and decryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 5, pp. 553–565, May 2022.

[43] D. Reis, J. Takeshita, T. Jung, M. Niemier, and X. S. Hu, "Computing-in-memory for performance and energy-efficient homomorphic encryption," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 28, no. 11, pp. 2300–2313, Nov. 2020.

[44] M. Imran, Z. Ul Abideen, and S. Pagliarini, "A systematic study of lattice-based NIST PQC algorithms: From reference implementations to hardware accelerators," 2020, *arXiv:2009.07091*.

[45] D. Reis, M. T. Niemier, and X. S. Hu, "A computing-in-memory engine for searching on homomorphically encrypted data," *IEEE J. Explor. Solid-State Comput. Devices Circuits*, vol. 5, pp. 123–131, 2019.

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.

[46] T. Kim, W. Jao, and S. Tjiang, "Circuit optimization using carry-save-adder cells," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 17, no. 10, pp. 974–984, Feb. 1998.
[47] Z. Lin et al., "In situ storing 8T SRAM-CIM macro for full-array Boolean logic and copy operations," *IEEE J. Solid-State Circuits*, vol. 58, no. 5, pp. 1472–1486, May 2023.
[48] E. Chu and A. George, *Inside the FFT Black Box: Serial and Parallel Fast Fourier Transform Algorithms*. Boca Raton, FL, USA: CRC Press, 1999.
[49] P. Longa and M. Naehrig, "Speeding up the number theoretic transform for faster ideal lattice-based cryptography," in *Proc. Cryp. Netw. Secu., Int. Conf. (CANS)*, Nov. 2016, pp. 124–139.
[50] H. Nejatollahi, S. Gupta, M. Imani, T. S. Rosing, R. Cammarota, and N. Dutt, "CryptoPIM: In-memory acceleration for lattice-based cryptographic hardware," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.
[51] Y. Zhao, X. Liu, Y. Hu, and H. Xiao, "Design of an efficient NTT/INTT architecture with low-complex memory mapping scheme," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 400–404, Jan. 2024.
[52] M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "OpenRAM: An open-source memory compiler," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, Nov. 2016, pp. 1–6.
[53] J. Chen et al., "AutoDCIM: An automated digital CIM compiler," in *Proc. 60th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2023, pp. 1–6.
[54] S. Song, W. Tang, T. Chen, and Z. Zhang, "LEIA: A 2.05 mm2 140 mW lattice encryption instruction accelerator in 40nm CMOS," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2018, pp. 1–4.
[55] G. Xin et al., "VPQC: A domain-specific vector processor for post-quantum cryptography based on RISC-V architecture," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 67, no. 8, pp. 2672–2684, Aug. 2020.
[56] H. Nejatollahi, S. Shahhosseini, R. Cammarota, and N. Dutt, "Exploring energy efficient quantum-resistant signal processing using array processors," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Barcelona, Spain, May 2020, pp. 1539–1543.
[57] W. Tan, Y. Lao, and K. K. Parhi, "KyberMat: Efficient accelerator for matrix-vector polynomial multiplication in CRYSTALS-kyber scheme via NTT and polyphase decomposition," in *Proc. IEEE/ACM Int. Conf. Comput. Aided Design (ICCAD)*, Oct. 2023, pp. 1–9.
[58] R. P. Rajput and M. N. S. Swamy, "High speed modified booth encoder multiplier for signed and unsigned numbers," in *Proc. UKSim 14th Int. Conf. Comput. Modeling Simulation*, Mar. 2012, pp. 649–654.
[59] B. Li et al., "A high-performance and low-cost Montgomery modular multiplication based on redundant binary representation," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 68, no. 7, pp. 2660–2664, Jul. 2021.



Xu Zhang received the B.S. degree in microelectronics science and engineering from Shandong University, Jinan, China, in 2014, and the M.S. degree in physical electronics from China Academy of Engineering Physics, Mianyang, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Electronic Science and Engineering, Nanjing University, Nanjing, China. His research interests include non-von Neumann architecture, memory circuit design, and hardware security.



Yaodong Wei received the B.S. degree in integrated circuit design and integrated system from Nanjing University, Nanjing, China, in 2024, where he is currently pursuing the M.S. degree in electronic information. His research interests include VLSI design and computing-in-memory.



Minghao Li received the B.S. degree in integrated circuit design and integrated system from Nanjing University, Nanjing, China, where he is currently pursuing the M.S. degree in electronic information. His research interests include digital signal processing and VLSI design.



Jing Tian (Member, IEEE) received the B.S. degree in microelectronics and the Ph.D. degree in information and communication engineering from Nanjing University, Nanjing, China, in 2015 and 2020, respectively. She is currently an Assistant Professor with Nanjing University. She has published more than 40 research peer-reviewed journals and conference papers, including more than ten papers at IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS (TCAS-I), IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS (TCAS-II), IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS (IEEE TVLSI), IEEE TRANSACTIONS ON COMPUTERS (IEEE TC), IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS (IEEE TCAD), and TCHES. Her research interests include optimized VLSI design for digital signal processing and cryptography, especially for post-quantum cryptography. She is a member of IEEE VLSI Systems and Applications Technical Committee. She was a recipient of Excellent Doctoral Dissertation of Nanjing University and Young Elite Scientist Sponsorship Program by CAST.



Zhongfeng Wang (Fellow, IEEE) received the B.E. and M.S. degrees from the Department of Automation, Tsinghua University, Beijing, China, in 1988 and 1990, respectively, and the Ph.D. degree from the University of Minnesota, Minneapolis, in 2000. He has been with Nanjing University, China, as a Distinguished Professor since 2016. Previously, he was with Broadcom Corporation, CA, USA, from 2007 to 2016, as a leading VLSI Architect. Before that, he was with Oregon State University and National Semiconductor Corporation. He is currently a World-Recognized Expert on Low-Power High-Speed VLSI Design for Signal Processing Systems. He has published over 200 technical articles with multiple best paper awards received from the IEEE technical societies, among which is the VLSI Transactions Best Paper Award of 2007. He has edited one book VLSI and held more than 20 U.S. and Chinese patents. His current research interests are in the area of optimized VLSI design for digital communications and deep learning. In the current record, he has had many articles ranking among top 25 most (annually) downloaded manuscripts in IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS. In the past, he has served as an Associate Editor for IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS (TCAS-I), IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS (TCAS-II), and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS (IEEE TVLSI) for many terms. He has also served as a TPC member and various chairs for tens of international conferences. Moreover, he has contributed significantly to the industrial standards. So far, his technical proposals have been adopted by more than 15 international networking standards. In 2015, he was elevated to a fellow of IEEE for contributions to VLSI design and implementation of FEC coding.

Authorized licensed use limited to: Indian Institute of Technology Gandhinagar. Downloaded on October 03,2026 at 08:23:01 UTC from IEEE Xplore. Restrictions apply.